STAGE 7: Advanced Flutter (Animations, Custom Widgets, Testing)

Flutter & Dart: From Zero to Expert — Stage Textbook 7 of 8

Goal: Add polish with animation, build your own reusable widgets, and prove correctness with tests.
Estimated time: 4–5 weeks
Prerequisite: Stage 6 (Networking & Local Storage)

7.1 Implicit Animations

```dart
class AnimatedBox extends StatefulWidget {
  const AnimatedBox({super.key});
  @override
  State<AnimatedBox> createState() => _AnimatedBoxState();
}

class _AnimatedBoxState extends State<AnimatedBox> {
  bool _expanded = false;

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: () => setState(() => _expanded = !_expanded),
      child: AnimatedContainer(
        duration: const Duration(milliseconds: 300),
        curve: Curves.easeInOut,
        width: _expanded ? 200 : 100,
        height: _expanded ? 200 : 100,
        color: _expanded ? Colors.blue : Colors.red,
      ),
    );
  }
}
```

Key rule: Widgets prefixed `Animated` (`AnimatedContainer`, `AnimatedOpacity`, `AnimatedAlign`, ...) animate automatically between old and new values whenever `setState()` changes one of their properties — no manual animation code required.

7.2 Explicit Animations with AnimationController

```dart
class PulsingDot extends StatefulWidget {
  const PulsingDot({super.key});
  @override
  State<PulsingDot> createState() => _PulsingDotState();
}

class _PulsingDotState extends State<PulsingDot> with SingleTickerProviderStateMixin {
  late AnimationController _controller;
  late Animation<double> _scale;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(vsync: this, duration: const Duration(seconds: 1))
      ..repeat(reverse: true);
    _scale = Tween<double>(begin: 0.8, end: 1.2).animate(_controller);
  }

  @override
  void dispose() {
    _controller.dispose(); // always dispose controllers
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return ScaleTransition(
      scale: _scale,
      child: Container(width: 40, height: 40, decoration: const BoxDecoration(shape: BoxShape.circle, color: Color
    );
  }
}
```

Key rules:

- `AnimationController` gives full manual control over an animation's timeline (start, stop, repeat, reverse).
- It must be disposed in `dispose()` — failing to do so is a common source of memory leaks.
- `SingleTickerProviderStateMixin` is required to provide the "ticker" that drives the controller's frames.

7.3 Custom Widgets and CustomPainter

```dart
class RatingStars extends StatelessWidget {
  final int rating; // 0–5
  const RatingStars({super.key, required this.rating});

  @override
  Widget build(BuildContext context) {
    return Row(
      children: List.generate(5, (i) => Icon(
        i < rating ? Icons.star : Icons.star_border,
        color: Colors.amber,
        size: 20,
      )),
    );
  }
}

// Drawing raw shapes when no widget fits your needs:
class ArcPainter extends CustomPainter {
  @override
  void paint(Canvas canvas, Size size) {
    final paint = Paint()
      ..color = Colors.blue
      ..strokeWidth = 6
      ..style = PaintingStyle.stroke;
    canvas.drawArc(Rect.fromLTWH(0, 0, size.width, size.height), 0, 3.14, false, paint);
  }

  @override
  bool shouldRepaint(covariant CustomPainter oldDelegate) => false;
}

// Usage: CustomPaint(size: const Size(100, 100), painter: ArcPainter())
```

Why build custom widgets: Wrapping repeated UI patterns (like `RatingStars`) into their own widget keeps `build()` methods short and makes the pattern reusable and independently testable. `CustomPainter` is your escape hatch when no built-in widget can draw what you need.

7.4 Unit Testing

Add to `pubspec.yaml` (dev_dependencies): `test: ^1.0.0` (included by default in Flutter projects)

```dart
// lib/calculator.dart
int add(int a, int b) => a + b;

// test/calculator_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:my_app/calculator.dart';

void main() {
  test('add() sums two integers correctly', () {
    expect(add(2, 3), 5);
  });

  test('add() handles negative numbers', () {
    expect(add(-2, 3), 1);
  });
}
```

Run with: `flutter test`

Key rule: Unit tests check pure logic (functions, classes) with no widgets or UI involved — they should be fast and have zero dependency on the screen.

7.5 Widget Testing

```dart
import 'package:flutter_test/flutter_test.dart';
import 'package:my_app/counter_widget.dart';

void main() {
  testWidgets('Counter increments when button is tapped', (tester) async {
    await tester.pumpWidget(const MaterialApp(home: Counter()));

    expect(find.text('Count: 0'), findsOneWidget);

    await tester.tap(find.byType(ElevatedButton));
    await tester.pump(); // rebuild after the state change

    expect(find.text('Count: 1'), findsOneWidget);
  });
}
```

Key rules:

- `tester.pumpWidget()` renders a widget inside a test environment.
- `tester.tap()` simulates a user tap; `tester.pump()` triggers a rebuild afterward.
- `find.text()` / `find.byType()` locate widgets to assert against.

7.6 Hands-On Project: Animated Quiz Card Widget

Build a reusable Flutter widget package that:

1. Defines a custom `FlipCard` widget using `AnimationController` and a `Transform` to flip between a "question" side and an "answer" side when tapped.
2. Defines a separate `ScoreBadge` widget showing a score out of 5, using `AnimatedContainer` to grow slightly whenever the score updates.
3. Has at least 2 unit tests for any pure scoring logic (e.g., a function that calculates percentage correct).
4. Has at least 1 widget test that taps the `FlipCard` and verifies the answer text becomes visible.

This project combines explicit animation, custom reusable widgets, and both layers of testing — all of Stage 7.

7.7 Stage 7 Test

Question 1: What is the key difference between implicit animations (`AnimatedContainer`) and explicit animations (`AnimationController`)?

Question 2: Why must an `AnimationController` always be disposed, and where do you do that?

Question 3: When would you reach for `CustomPainter` instead of composing existing widgets?

Question 4: What is the difference between a unit test and a widget test in Flutter?

Question 5: What does `tester.pump()` do after `tester.tap()` in a widget test, and why is it necessary?

Passing score: Get all 5 correct before moving to Stage 8.